

1. ¿Qué problema resuelve Singleton aquí?

El patrón Singleton es clave para mantener una configuración centralizada en todos los componentes. Por ejemplo, si fijamos el umbral de CPU en 80 %, no queremos que cada monitor tenga un ajuste distinto, ya que podríamos terminar con un microservicio usando 80 %, otro 90 % y otro cualquier valor. Por eso, en **MonitoringConfig** aplicamos Singleton, asegurando una única **instancia** compartida de la configuración. Así, si un componente cambia el umbral, los demás consultan ese mismo valor, evitando configuraciones independientes y garantizando uniformidad en todo el sistema.

2. ¿Cómo desacoplar servicios externos?

En el ejercicio desacoplamos los servicios externos principalmente usando Adapter y Observer. Con Adapter podemos integrar una API externa aunque tenga una estructura distinta a la que maneja nuestro sistema. Por ejemplo, la API antigua devuelve `cpu_usage_percent`, mientras que nuestro sistema utiliza `get_cpu_usage()`. El Adapter se encarga de hacer esa traducción.

Por otro lado, usamos Observer para desacoplar el monitor de los distintos canales de alerta. El monitor no necesita conocer el funcionamiento de Email, Slack, SMS o Teams; simplemente notifica a los observadores registrados.

3. ¿Por qué Facade mejora mantenibilidad?

El patrón Facade mejora la mantenibilidad al ocultar la complejidad interna del sistema y ofrecer un punto de acceso sencillo. En nuestro caso, contamos con varios componentes como el monitor, los observadores, el Adapter y la configuración. En lugar de que `main.py` tenga que conocer y crear todos estos objetos, utilizamos **MonitoringFacade**, que se encarga de coordinarlos. Si en el futuro cambia alguna parte interna, podremos hacer las modificaciones dentro del subsistema sin afectar demasiado al código que usa la fachada.

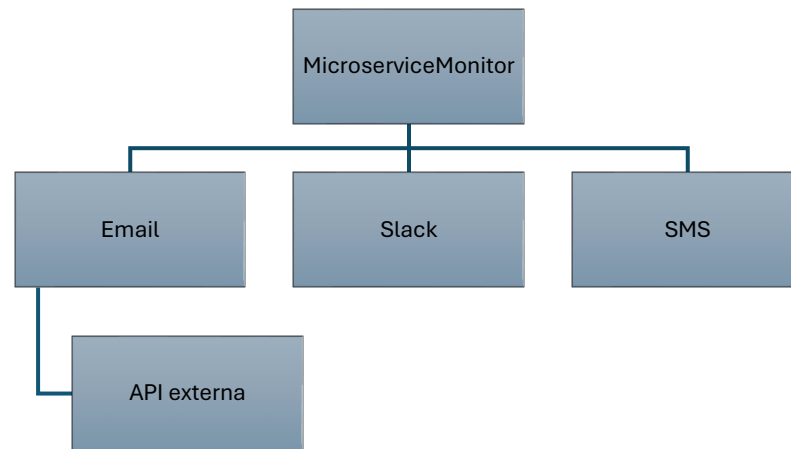
4. ¿Cómo escalar el sistema a 100 microservicios?

Para escalar a 100 microservicios, reutilizaríamos la misma lógica de monitoreo y los mismos componentes. Cada microservicio se registraría para ser monitoreado y emplearía los mismos mecanismos para obtener métricas, consultar la configuración y generar alertas. Si alguno usara una API distinta, crearíamos un nuevo Adapter; y si necesitáramos un canal de alerta adicional, desarrollaríamos otro Observer. Así evitaríamos duplicar toda la lógica en cada microservicio.

5. ¿Dónde se produciría alto acoplamiento si no se aplican patrones?

El alto acoplamiento ocurriría principalmente si **MicroserviceMonitor** incluyera directamente la lógica de cada canal de alerta y, además, conociera los detalles de las APIs externas. Por ejemplo, si dentro del monitor hubiera código específico para enviar Email, Slack, SMS y Teams, cada vez que quisiéramos agregar un nuevo canal tendríamos que modificar esa clase. Lo mismo pasaría si el monitor dependiera de una API externa específica y dicha API cambiara.

Por ejemplo, si imaginamos esto, el monitor estaría encargado de demasiadas cosas.



Y si lo hacemos con los patrones, quedaría así

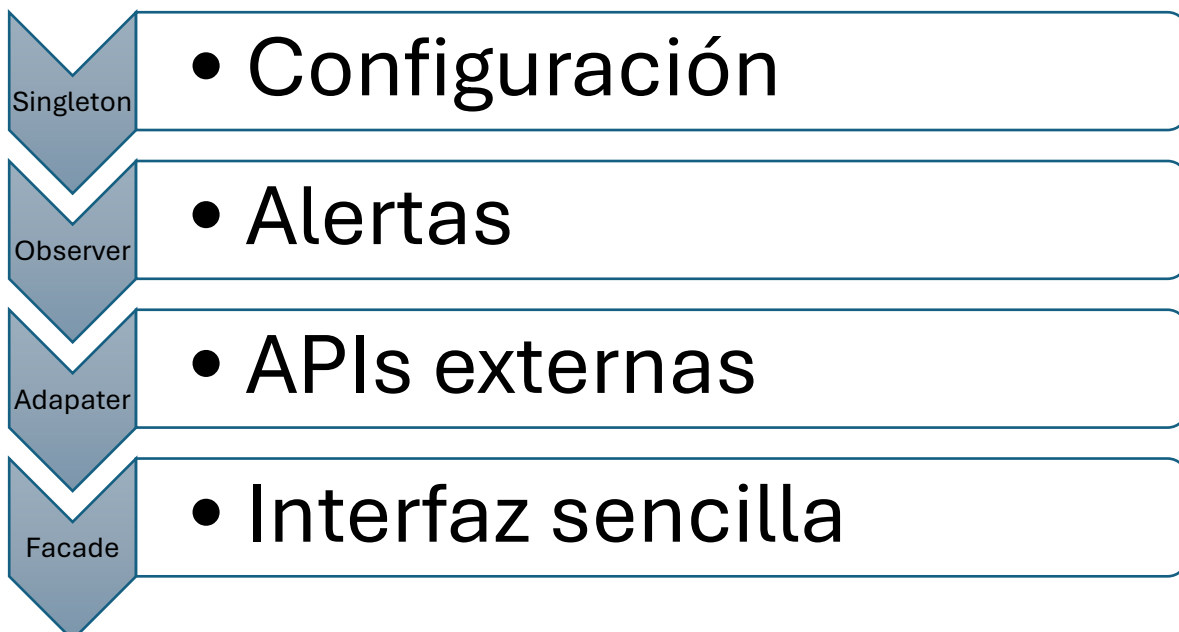
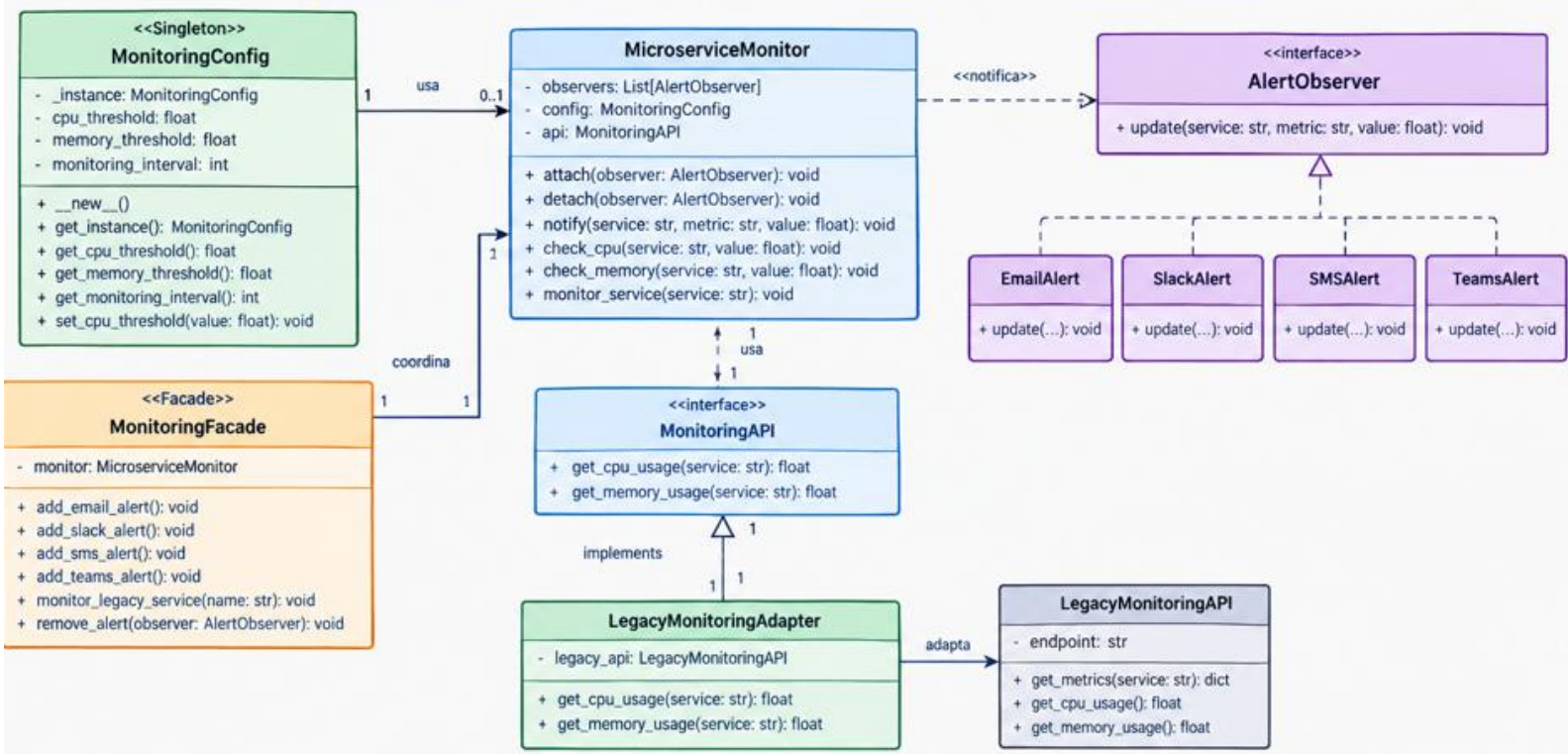


Diagrama UML – Sistema de Monitoreo de Microservicios



Relaciones UML

- Asociación (usa / coordina)
- - - - -> Dependencia (notifica / adapta)
- |> Herencia (implementa)
- - - - -|> Realización (implementa interfaz)
- 1 0..1 Multiplicidad

Patrones de diseño aplicados

- Singleton** `MonitoringConfig`: una sola instancia de configuración.
- Facade** `MonitoringFacade`: interfaz sencilla para usar el sistema.
- Observer** `AlertObserver`: permite diferentes canales de alerta sin acoplarlos.
- Adapter** `LegacyMonitoringAdapter`: adapta la API externa al formato del sistema.

Resumen de la arquitectura

- `MonitoringFacade` coordina todos los componentes.
- `MicroserviceMonitor` obtiene métricas, compara con umbrales y notifica a los observadores.
- `MonitoringConfig` mantiene una configuración centralizada (Singleton).
- Los observadores (Email, Slack, SMS, Teams) reciben las alertas (Observer).
- `LegacyMonitoringAdapter` traduce la API antigua al formato esperado (Adapter).